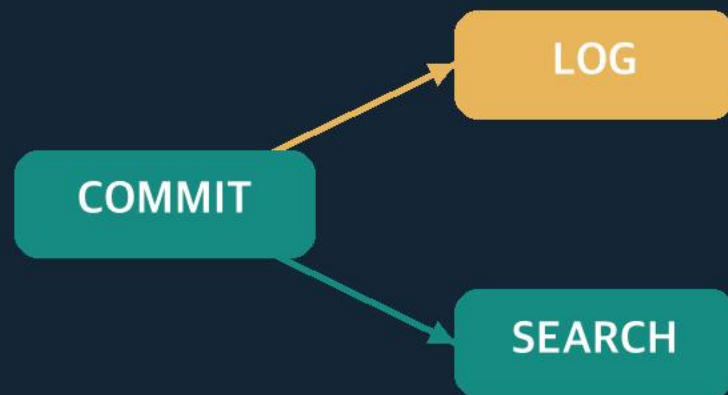


Mini Git

실행부터 평가 설명까지

직접 실행하고, 근거로 확인하고, 내 말로 설명하기



과제

B3-2 · 자료구조와 알고리즘

검증

Python 3.12.13 · 테스트 15개 통과

대상

처음 배우는 학습자 · 평가 발표 준비

상세 설명

이 자료는 AI-SW-Basic/b3-2의 실제 구현과 실행 기록을 바탕으로 만들었습니다. 순서는 실행, 검증, 원리 설명입니다. Mini Git은 파일 내용을 관리하는 완전한 Git이 아니라 커밋 메타데이터로 그래프·탐색·정렬·검색을 배우는 프로그램입니다.

발표에서 답할 세 가지

01 어떻게 실행했나요?

b3-2 폴더에서
python main.py
명령을 한 줄씩 입력합니다.

02 어떻게 검증하나요?

화면의 결과를 확인하고
자동 테스트를 실행합니다.
정상·오류·경계 입력을 비교합니다.

03 근거는 어디 있나요?

코드: mini_git/
테스트: tests/
기록: 제출/results/

상세 설명

프로그램은 실행된다는 사실만으로 정답이라고 할 수 없습니다. 과제의 조건을 결과와 연결해야 합니다. 예를 들어 LOG는 출력이 있는지뿐 아니라 부모가 먼저 나오는지 봐야 합니다. 오늘의 세 가지 질문은 실행 방법, 확인 기준, 증거 위치입니다.

명령 → 기대 결과 → 근거 파일을 연결해서 설명합니다.

먼저 폴더의 역할을 구분하세요

AI-SW-Basic/b3-2/

main.py	실행 시작점
README.md	실행 설명
mini_git/	실제 구현
cli.py / repository.py	입력·상태 관리
traversal.py	그래프 탐색
sorting.py / index.py	정렬·검색
tests/test_mini_git.py	자동 검사
제출/실행결과.md	결과 안내
제출/results/	원본 결과

상세 설명

모든 파일 경로는 b3-2를 기준으로 말합니다. main.py만 전달하면 내부 패키지를 찾을 수 없으므로 mini_git 폴더도 함께 있어야 합니다. 과제 원문은 요구사항, 최종검수결과는 요구사항별 판단, 평가질문_설명자료는 답변 연습 자료입니다. results의 텍스트는 실행 흔적이며 테스트 소스와 구분합니다.

README는 사용 설명서, tests는 검사 도구, results는 검사 기록입니다.

실행은 이 세 줄로 시작합니다

터미널에 입력

```
cd ~/AI-SW-Basic/b3-2  
python3 --version  
python3 main.py
```

정상 화면

Mini Git CLI — 종료하려면 exit 또는 quit을 입력하세요.

```
mini-git>
```

알아두기

python 명령이 설정되어 있으면 python main.py도 같습니다.

추가 라이브러리 설치의 필요 없습니다.

종료: exit 또는 quit

상세 설명

터미널은 프로그램을 시작하는 장소입니다. 시작 뒤 mini-git>가 보이면 그때부터는 Mini Git 명령을 입력합니다. python과 python3는 컴퓨터 설정에 따라 이름이 다릅니다. 이번 검증에 사용한 버전은 Python 3.12.13입니다. 표시된 경로가 다르면 실제 프로젝트 위치로 cd 하세요.

Python 3.10 이상인지 확인하고, b3-2 안에서 실행합니다.

브랜치가 갈라지는 상황을 만들어 봅니다

mini-git> 뒤에 한 줄씩 입력

```
init "Alice Kim"  
commit "Initial commit"  
branch feature  
switch feature  
commit "Add login feature"  
switch main  
commit "Add payment feature"  
log
```

세 커밋의 관계

Initial: 부모 없음

Login: Initial이 부모

Payment: Initial이 부모

main → Payment

feature → Login

상세 설명

첫 커밋을 만든 다음 feature를 만듭니다. feature는 그 순간의 커밋을 가리키는 이름표입니다. feature에서 로그인 커밋을 만들고 main으로 돌아 오면 main은 여전히 Initial을 가리킵니다. 그래서 결제 커밋도 Initial을 부모로 합니다. branch는 전환하지 않으며 switch가 전환합니다. 사용자명·메시지에 공백이 있으면 따옴표를 씁니다.

COMMIT은 현재 브랜치의 끝을 부모로 삼고, 그 브랜치의 끝을 이동합니다.

출력에 무엇이 보이면 성공일까요?

명령 / 기대 결과 / 확인 기준

INIT / main·사용자 표시 / 초기 상태 생성

COMMIT / [branch hash] message / hash와 메시지 표시

LOG / 세 커밋 출력 / Initial이 두 자식보다 먼저

SEARCH login / 1개 / Login 메시지만 포함

SEARCH --author="Alice Kim" / 3개 / 작성자 정확히 일치

LOG --sort-by=date / 날짜 오름차순 / timestamp 비교

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 후

LOG → SEARCH login → SEARCH --author="Alice Kim" → LOG --sort-by=date

확인: 5페이지의 새 실습 기준: 커밋 3개, login 검색 1개, 작성자 검색 3개, 날짜 오름차순

hash는 커밋의 식별자이지 외워야 할 정답이 아닙니다. 실제 화면에 나온 hash를 사용해야 합니다. 로그에서는 hash, author, timestamp, message가 식별 가능해야 합니다. 한 세션에서는 작성자가 한 명이므로 author 정렬의 순서 변화가 눈에 보이지 않을 수 있습니다. 서로 다른 작성자의 정렬은 테스트에서 별도로 확인합니다.

문구보다 동작의 규칙을 확인합니다. hash는 실행마다 달라질 수 있습니다.

PATH와 ANCESTORS는 실제 hash로 확인합니다

최단 경로 확인 — PATH

PATH 5fa901 837f82

기대 경로:

5fa901 → 0a7990 → 837f82

최단 거리 = 간선 2개

조상 확인 — ANCESTORS

ANCESTORS 837f82

기대 조상: 0a7990

ANCESTORS 0a7990

기대: No ancestors.

먼저 LOG에서 hash 확인

LOG

0a7990: 최초 커밋

5fa901: 로그인 커밋

837f82: 결제 커밋

세션이 바뀌면 실제 hash로 교체

상세 설명

mini-git> 프롬프트에서 LOG로 hash를 확인한 뒤 PATH 5fa901 837f82와 ANCESTORS 837f82를 입력합니다. 이 명령은 사용자가 제시한 실행 로그의 hash를 사용한 예시이며, 로그인과 결제 커밋이 최초 커밋을 공통 부모로 갖는 흐름을 기준으로 합니다. 새 세션에서는 hash가 달라지므로 현재 LOG의 값으로 바꿉니다. PATH는 Login → Initial → Payment로 간선 두 개를 지나며, 화살표는 경로 순서입니다. ANCESTORS는 부모 방향만 따라가고 자신은 제외합니다. ANCESTORS 0a7990은 No ancestors.를 출력합니다. PATH가 자기 자신이면 간선 수는 0입니다. 경로가 없으면 No path, 존재하지 않는 hash면 Unknown commit입니다.

PATH는 양방향으로 길을 찾고, ANCESTORS는 부모 방향만 따라갑니다.

자동 검증은 한 명령으로 재현합니다

b3-2 터미널에서 실행

```
python3 -B -m unittest discover -s tests -v
```

포함된 검사

무작위 DAG 50개 · 정렬 데이터 최대 1,000개
부모 5,000개 · hash 강제 충돌
실제 REPL 실행 · exit/quit 종료

15

TESTS PASSED

Ran 15 tests
OK

상세 설명

unittest는 Python에 들어 있는 테스트 실행 도구입니다. -s tests는 tests 폴더를 검사하라는 뜻이고 -v는 검사 이름을 자세히 보여줍니다. -B는 파이썬 캐시 파일 생성을 막는 선택 옵션입니다. 각 test 함수는 여러 assert와 반복 입력을 포함합니다. FAIL 또는 ERROR가 나오면 해당 이름과 마지막 오류 메시지를 읽어야 합니다. tests의 코드는 검사 방법이며 results/11은 검사 기록입니다.

15개 테스트는 여러 입력을 반복 확인합니다. 성공 표시는 OK입니다.

평가자에게 보여 줄 근거 위치

확인하고 싶은 것 / 파일

실행 안내 / README.md, 가이드.md

요구사항별 판단 / 최종검수결과.md

실행 결과 요약 / 제출/실행결과.md

기본 기능 원본 / 제출/results/01~09_*.txt

보완 동작 원본 / 제출/results/10_보완_회귀검증.txt

테스트 원본 결과 / 제출/results/11_자동검증.txt

검사 소스 / tests/test_mini_git.py

알고리즘 구현 / mini_git/traversal.py, sorting.py, index.py

상세 설명

01~09 및 10 중 09를 제외한 기록은 dispatch 함수에 실제 명령을 전달한 출력입니다. 09는 main.py 별도 프로세스 종료 기록입니다. 테스트 안에서도 main.py를 직접 실행합니다. 따라서 모든 텍스트를 화면 캡처라고 부르지 않습니다. 오늘 PPT와 발표 노트는 제출/발표자료에 저장합니다.

결과 → 검사 방법 → 구현 순서로 파일을 열면 설명하기 쉽습니다.

검수에서 발견한 문제를 이렇게 고쳤습니다

문제 → 수정 → 확인

반복 단어가 커밋 중복 출력 → 커밋별 토큰 중복 제거 → 1개만 반환
공백 포함 검색어 0건 → 연속 토큰 구문 검색 → login feature 검색
기존 BRANCH 덮어쓰기 → Invalid args로 거부 → 모든 HEAD 보존
조상 큐 pop(0) 비용 → 인덱스로 읽기 → 다중 부모 검증
공백뿐인 입력 수용 → .strip() 검사 → 표준 오류 반환
버전·hash 설명 부정확 → 실제 환경·충돌 재시도 설명 → 문서 갱신

실행 확인 · 명령은 한 줄씩 입력

일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k phrase_order
```

확인: 연속 구문·반복 토큰·대소문자·공백 처리 검증 · 마지막 OK 확인

회귀 테스트는 한 번 고친 문제가 다시 생기지 않게 하는 검사입니다. 중복 검색은 메시지 login login LOGIN으로 확인합니다. 검색 결과는 단어 등장 횟수가 아니라 커밋 개수입니다. branch main을 다른 브랜치에서 호출해도 main이 이동하지 않아야 합니다. Python 3.9.6의 과거 검수 대신 3.12.13 검증을 기준으로 설명합니다.

기존의 전체 통과 결론을 재검수하고, 오류를 고친 뒤 테스트를 추가했습니다.

입력과 알고리즘을 나눈 이유

입력·출력

main.py → cli.py
문자열 파싱·명령 선택·결과 출력



저장소 상태

repository.py
사용자·브랜치·HEAD·커밋 관리



알고리즘

traversal.py: 탐색
sorting.py: 정렬
index.py: 역색인

실행 확인 · 명령은 한 줄씩 입력

일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k actual_entrpoint
```

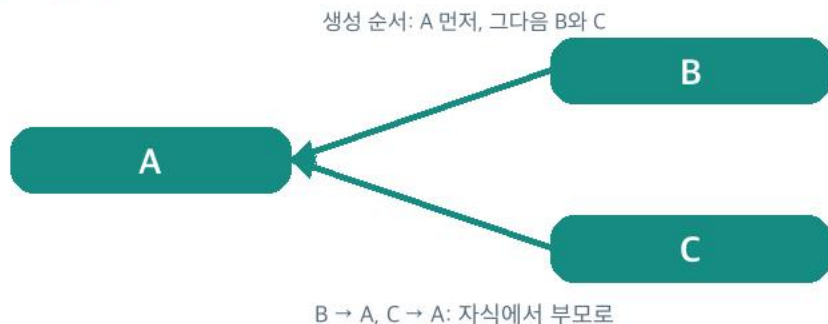
확인: main.py 실행과 실제 CLI 입출력·종료 검증 · 마지막 OK 확인

초보자가 코드를 읽을 때는 실행 시작점부터 따라가면 됩니다. cli는 입력 문장을 해석하고 repository 메서드를 호출합니다. repository는 상태를 바꾸거나 알고리즘에 조회를 부탁드립니다. 이를 함수와 클래스로 나누는 것은 과제의 필수 제약이기도 합니다. 테스트는 화면을 거치지 않고 알고리즘을 직접 호출할 수 있습니다.

입출력, 저장소 상태, 알고리즘을 나누면 따로 검증할 수 있습니다.

커밋 그래프는 왜 DAG인가요?

한눈에 이해하기



DAG란?

Directed Acyclic Graph
방향성 비순환 그래프
화살표 방향을 따라 돌아오는 고리가 없음

왜 방향성인가요? — Directed

노드 = 커밋, 간선 = 부모 참조
자식 → 부모 방향으로 연결
B → A는 B의 부모가 A라는 뜻

왜 비순환인가요? — Acyclic

새 커밋은 먼저 만들어진 부모만 참조
기존 커밋의 부모 참조는 유지
계속 이전으로 가므로 출발점에 못 돌아옴

상세 설명

평가 답변: DAG는 Directed Acyclic Graph의 약자로, 방향성 비순환 그래프입니다. 커밋 하나가 노드이고, 자식 커밋에서 부모 커밋으로 향하는 참조가 간선이므로 방향성이 있습니다. 새 커밋은 이미 만들어진 커밋만 부모로 참조하며, 기존 커밋의 부모 참조는 바꾸지 않습니다. 따라서 부모를 한 번 따라갈 때마다 생성 순서가 반드시 더 이전으로 갑니다. 더 이전으로만 이동해서는 출발 커밋으로 돌아올 수 없으므로 순환이 없습니다. 이 두 조건을 만족하여 커밋 그래프는 DAG입니다. 그림 설명: A를 먼저 만들고, A를 부모로 하는 B와 C를 만듭니다. B → A와 C → A는 가능하지만 A가 나중에 생긴 B나 C를 부모로 참조하지는 않습니다. B → A → B처럼 화살표를 따라 출발점으로 돌아오는 경로가 있으면 순환이며 DAG가 아닙니다. 주의: 여기서 이전은 timestamp 값이 아니라 생성 순서를 뜻합니다. PC 시계가 바뀌어도 부모는 생성 당시 존재한 커밋입니다. 병합으로 두 경로가 합쳐져도, 새 병합 커밋이 기존 커밋들을 부모로 참조하고 기존 부모 참조가 유지되면 방향성 순환은 생기지 않습니다. DAG는 반드시 트리일 필요가 없습니다. Mini Git의 정상 COMMIT 명령은 기존 HEAD를 부모로 저장하며 기존 커밋의 parents를 수정하지 않습니다. Commit 클래스 자체는 변경 가능한 Python 객체이므로 외부 코드가 parents를 임의로 변경하는 상황까지 이 생성 규칙이 보장하는 것은 아닙니다. CLI MERGE는 구현하지 않았고, 다중 부모는 자료구조와 테스트에서 지원합니다.

부모를 따라갈수록 생성 순서가 더 이전으로 가므로, 출발 커밋으로 돌아오는 순환이 없습니다.

브랜치와 HEAD는 '끝을 가리키는 이름표'

한눈에 이해하기



현재 브랜치: feature

A에서 feature 생성

main → A
feature → A
현재 브랜치: main

feature에서 B 커밋

main → A
feature → B
B.parents = [A.hash]

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습과 별도로 처음부터 실행

INIT "Alice Kim" → COMMIT "Initial commit" → BRANCH feature → SWITCH feature → COMMIT "Add login feature" → SWITCH main → LOG

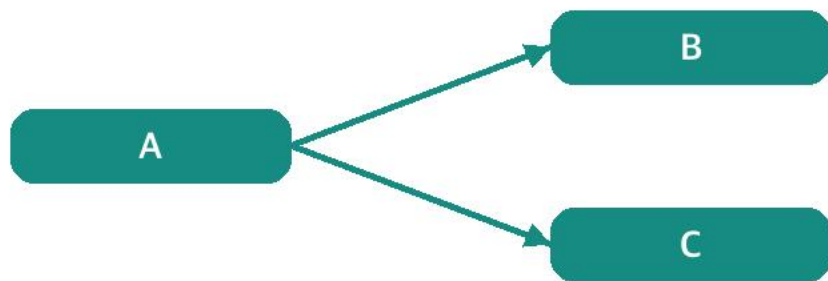
확인: INIT은 기존 데이터를 초기화함. main에는 최초 커밋, feature에는 로그인 커밋

HEAD를 커밋 하나를 담는 별도 객체라고 생각할 필요는 없습니다. 이 구현에서는 current_branch가 현재 브랜치를 나타내고 branches[current_branch]가 현재 커밋입니다. branch는 저장된 커밋을 복제하지 않습니다. 같은 커밋을 가리키는 이름표를 추가합니다. COMMIT은 현재 브랜치만 앞으로 이동시킵니다.

BRANCH는 이름표를 만들고, SWITCH는 사용할 이름표를 고릅니다.

LOG: 선수과목을 먼저 출력하는 방식

한눈에 이해하기



A를 먼저 출력 → B와 C가 후보

Kahn 알고리즘의 세 단계

1. 남은 부모 수를 센다: $A=0, B=1, C=1$
2. A를 출력하고 B·C의 수를 0으로 줄인다
3. 큐에 들어온 B·C를 차례로 출력한다

출력 예

$A \rightarrow B \rightarrow C$

A는 반드시 두 자식보다 먼저

B·C 사이의 순서는 FIFO 큐

방향 구분

저장 참조: 자식 → 부모

LOG 계산: 부모 → 자식으로 뒤집어 처리

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 후

LOG

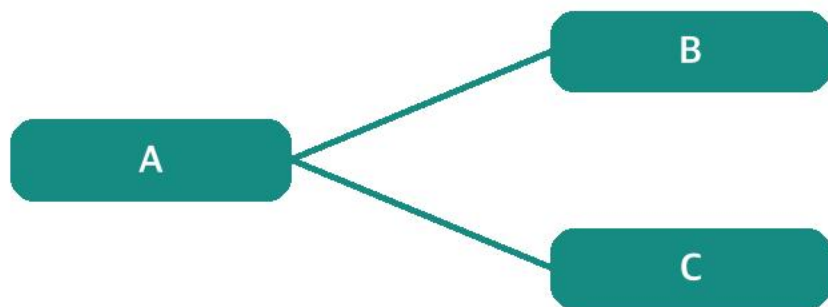
확인: Initial이 Login·Payment보다 먼저 출력

진입차수라는 어려운 용어는 아직 처리되지 않은 선수과목 수라고 풀어 설명합니다. FIFO는 먼저 들어온 것이 먼저 나오는 줄입니다. 동시에 후보가 된 커밋은 이 큐 순서를 따릅니다. 최신순 정렬은 요구사항을 만족하지 않습니다. 위상 정렬은 날짜가 아니라 관계를 사용하므로 부모 우선을 보장합니다. 시간복잡도는 V 개 커밋과 E 개 연결을 처리하는 $O(V+E)$ 입니다.

아직 출력하지 않은 부모가 0개인 커밋부터 꺼냅니다.

PATH: 한 칸씩 넓혀 가면 가장 짧은 길

한눈에 이해하기



B ↔ A ↔ C: 간선 2개

BFS = 가까운 곳부터

출발지: 거리 0
바로 이웃: 거리 1
그 이웃의 이웃: 거리 2

이 과제의 연결 규칙

부모·자식 연결은 양방향
Login ↔ Initial ↔ Payment
Login → Payment의 거리 = 2

경로가 없다면

두 커밋은 있지만 연결 안 됨: No path
hash 자체가 없음: Unknown commit: <hash>

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 후

PATH 5fa901 837f82 → PATH 5fa901 5fa901

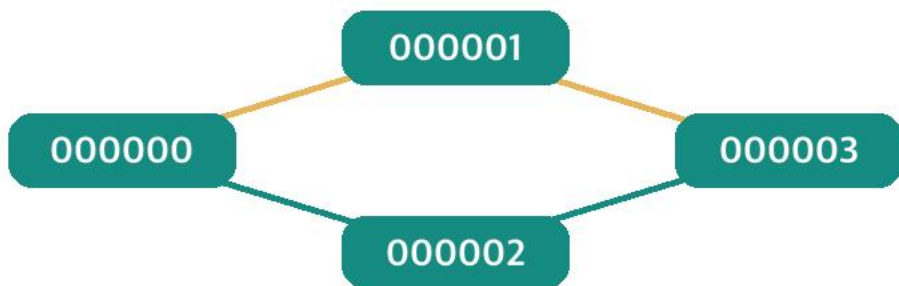
확인: 현재 LOG의 hash로 교체. 첫 경로는 간선 2개, 자기 자신은 간선 0개

BFS는 Breadth First Search, 너비 우선 탐색입니다. 물결이 가까운 곳부터 퍼진다고 생각하면 됩니다. 간선마다 비용이 같으므로 먼저 발견한 단계가 최단 거리입니다. PATH는 Git 부모 참조를 그대로 한 방향으로 따라가는 기능이 아니라 과제가 정한 무방향 경로입니다. 같은 출발지와 목적지는 거리 0입니다.

모든 연결의 비용이 1일 때 BFS는 간선 수가 가장 적은 거리를 찾습니다.

최단 경로가 둘이면 hash 사전순으로

한눈에 이해하기



테스트용 다중 부모 그래프

같은 거리의 두 경로

000000 → 000001 → 000003

000000 → 000002 → 000003

두 경로 모두 간선 2개

선택

000001 < 000002

첫 경로의 연결 문자열이 사전순 최소

구현 방법

출발·도착에서 각각 BFS

남은 거리가 1 줄어드는 후보만 선택

그중 hash가 가장 작은 다음 커밋으로

실행 확인 · 명령은 한 줄씩 입력

일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k merge_ancestors
```

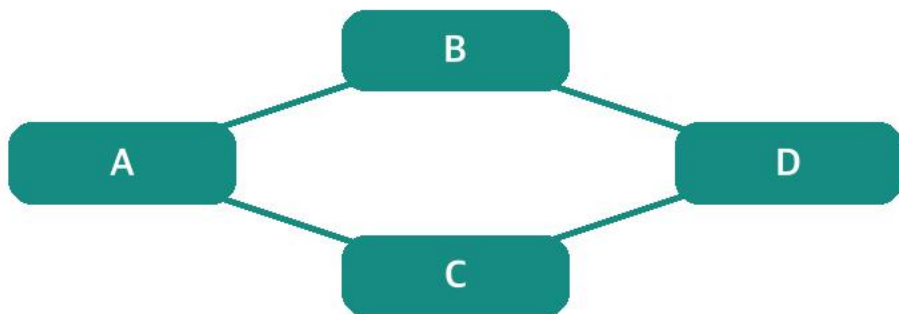
확인: CLI에 MERGE는 없음. 테스트용 다중 부모 그래프에서 사전순 최단 경로 확인 · 마지막 OK 확인

이 그림은 부모가 둘인 병합 노드를 포함한 테스트용 다이아몬드 그래프입니다. CLI에서 merge를 실행해서 만든 것이 아닙니다. dStart[next]=dStart[current]+1과 dEnd[next]=dEnd[current]-1 조건을 확인합니다. 작은 hash만 보고 아무 길이나 선택하면 더 긴 경로로 빠질 수 있으므로 최단 조건이 먼저입니다. 모든 최단 경로를 열거하지 않고 두 BFS와 경로 복원으로 계산합니다.

최단 거리 조건을 지키는 후보 중 다음 hash가 작은 쪽을 고릅니다.

ANCESTORS: 부모의 부모를 빠짐없이 찾기

한눈에 이해하기



테스트용 다중 부모 그래프

예: D의 부모가 B·C

B의 부모: A

C의 부모: A

D의 조상: B, C, A

A는 한 번만, D는 제외

visited = 방문 기록

set으로 이미 방문한 hash를 확인

중복으로 만난 조상은 건너뛰

큐는 앞을 삭제하지 않고 인덱스로 읽음

복잡도

$O(V+E)$

V: 도달 가능한 조상 수

E: 살펴보는 부모 연결 수

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 후

ANCESTORS 837f82 → ANCESTORS 0a7990

확인: 현재 LOG의 hash로 교체. Payment의 조상은 Initial, Initial은 No ancestors.

가족의 조상 목록을 만드는 것과 비슷하지만 여러 부모 경로가 같은 조상에 합쳐질 수 있습니다. set은 이미 본 hash를 빠르게 확인하는 집합입니다. 초기 큐에는 조회 대상의 부모만 넣으므로 자신은 결과에 들어가지 않습니다. 이전의 list.pop(0)은 원소들을 당겨야 했고, 지금은 head 인덱스를 이동하여 큐 비용을 줄였습니다.

부모 방향만 탐색하고, 이미 본 조상은 한 번만 포함합니다.

정렬은 병합 정렬을 직접 구현했습니다

한눈에 이해하기

[4, 1, 3, 2]

[4, 1] [3, 2]

[4] [1] [3] [2]

[1, 4] [2, 3]

[1, 2, 3, 4]

나누기

[4, 1, 3, 2]

[4, 1] [3, 2]

[4] [1] [3] [2]

합치기

[1, 4] [2, 3]

[1, 2, 3, 4]

평가 질문: 왜 퀵 정렬이 아닌가요?

병합 정렬은 평균·최악 $O(n \log n)$

같은 값의 원래 순서를 지키는 안정 정렬

일반적 퀵 정렬은 최악 $O(n^2)$, 보통 불안정

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 후

LOG --sort-by=date → LOG --sort-by=author

확인: 날짜·작성자 오름차순. 한 작성자만 있으면 author 정렬의 변화가 보이지 않을 수 있음

표준 sorted와 list.sort를 사용하는 것은 금지되어 있습니다. key 함수는 비교할 값을 꺼내는 도구입니다. date에서는 timestamp, author에서는 author를 꺼냅니다. 일반적인 병합 정렬은 임시 배열 공간 $O(n)$ 을 사용합니다. 퀵 정렬이 언제나 나쁜 것은 아니며, 이 과제에서는 최악 시간과 안정성이 설명하기 쉬운 병합 정렬을 선택했습니다.

반으로 나누고, 작은 값부터 합칩니다. 날짜·작성자는 비교 키만 바꿉니다.

$O(n \log n)$ 과 안정 정렬을 쉬운 말로

작업량

n = 데이터 개수

8개를 나누면 $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

각 병합 층의 전체 작업은 약 n

평균·최악 시간: $O(n \log n)$

안정 정렬

정렬 전: Bob, Alice①, Alice②

정렬 후: Alice①, Alice②, Bob

키가 같으면 왼쪽을 먼저 채택: $\leq$

①·②는 설명용 순서 표시

실행 확인 · 명령은 한 줄씩 입력

일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k sort_keys
```

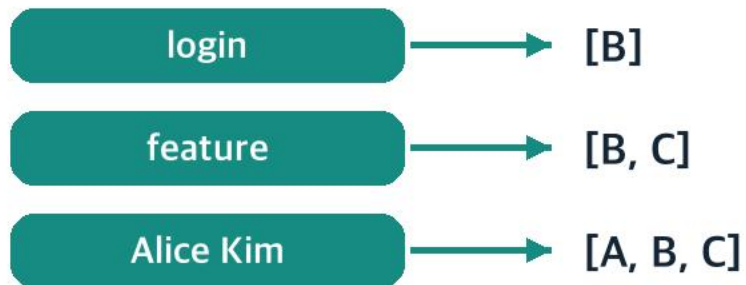
확인: 정렬 키·안정성·원소 보존·원본 불변 확인 · 마지막 OK 확인

시간복잡도는 실제 초 단위 실행 시간이 아니라 입력이 커질 때 작업량이 얼마나 늘어나는지 표현합니다. 안정성은 프로그램이 안 죽는다는 뜻이 아니라 같은 비교값끼리 원래 순서를 지킨다는 뜻입니다. $\leq$ 를 쓰면 두 값이 같을 때 원래 앞에 있던 왼쪽 원소를 먼저 꺼냅니다. 테스트는 순서뿐 아니라 원본 목록이 바뀌지 않았는지도 확인합니다.

한 층에서 n 개를 다루고 층은 $\log_2 n$ 개. 같은 키는 원래 순서를 보존합니다.

역색인: 책 뒤의 '찾아보기'

한눈에 이해하기



메시지를 토큰으로

Add Login Feature
→ add, login, feature
공백 split + 소문자 lower

색인 예

login → [B]
feature → [B, C]
Alice Kim → [A, B, C]

검색 비용

전체 순회: 커밋 수 N에 비례
단어·작성자 색인 조회: 평균 O(1)
결과 k개를 복사·출력: O(k)

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 후

SEARCH login → SEARCH feature → SEARCH --author="Alice Kim"

확인: 5페이지의 새 실습 기준: 각각 1개, 2개, 3개

역색인의 역은 커밋에서 단어를 찾는 대신 단어에서 커밋을 찾는다는 의미입니다. 키워드를 담은 dict와 작성자를 담은 dict 두 종류를 사용합니다. 한 메시지의 login이 세 번 반복되어도 그 커밋 hash는 해당 목록에 한 번만 들어갑니다. 쓰기 시점의 색인 비용과 추가 메모리 대신 검색 속도를 얻습니다. 해싱과 정규화는 문자열 길이만큼 비용이 별도로 듭니다.

커밋을 만들 때 단어별 커밋 목록을 준비해서, 검색 때 바로 꺼냅니다.

공백 포함 검색은 '연속 구문'으로

SEARCH "login feature"

Add Login Feature → 일치
login another feature → 불일치
feature login → 불일치
대소문자·공백 수는 무시

후보 선택과 비용

login 목록과 feature 목록 중 작은 목록 선택
후보의 토큰이 연속해서 같은지 확인
구문 길이 q , 후보 토큰 합 T
후보 검사: 최악 $O(T \cdot q)$

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 이후 추가 입력
COMMIT "login another feature" → COMMIT "feature login" → SEARCH "login feature"
확인: Add login feature만 일치. 추가 두 커밋은 단어가 연속하지 않거나 순서가 반대여서 제외

이 검색은 부분 문자열 검색이 아닙니다. login으로 logging을 찾지 않습니다. 여러 단어를 검색할 때 단어가 다 있다는 사실만으로는 부족합니다. 붙어 있는지, 순서가 맞는지, 같은 단어를 반복한 구문인지 확인해야 합니다. 모든 커밋에 흔한 단어라면 후보가 많아질 수 있으므로 모든 검색이 $O(1)$ 이라고 말하지 않습니다.

드문 단어의 후보만 확인하므로 전체 저장소를 훑지 않습니다.

평가 질문: dict와 hash를 왜 썼나요?

Q. list 대신 dict인 이유?

list: 같은 hash를 처음부터 찾아야 함 $\rightarrow O(N)$

dict: hash를 키로 평균 $O(1)$ 조회

Repository.commits[hash]로 바로 찾을 수 있음

Q. hash 중복은 어떻게 막나요?

내용 + 시퀀스를 SHA-1으로 해싱

앞 6자리를 기존 hash와 비교

충돌이면 새 시퀀스로 다시 시도

카운터만으로 충돌이 사라지지 않는다는 점

실행 확인 · 명령은 한 줄씩 입력

일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k hash_collision
```

확인: hash 충돌을 강제로 발생시켜 재시도와 식별자 중복 방지 확인 · 마지막 OK 확인

hash는 이 구현에서 짧은 문자열 식별자입니다. 같은 내용을 다시 커밋해도 시퀀스가 바뀌어 새로운 입력으로 해싱합니다. 서로 다른 입력의 출력이 같을 수 있으므로 존재 여부 검사와 재시도가 핵심입니다. 6자리 공간은 16의 6승으로 유한하다는 한계도 있습니다. 강제로 같은 digest가 나오게 만든 테스트로 재시도를 확인했습니다.

dict는 빠른 조회를 맡고, hash 중복 검사는 식별자의 유일성을 지킵니다.

평가 질문: CLI 정책을 설명해 보세요

질문 / 실제 동작과 이유

LOG의 [branch]는? / 생성 당시 브랜치 표시, 현재 HEAD 표시는 아님

정렬 LOG에도 표시? / 필수 필드 외에 같은 출력 형식을 유지

INIT을 다시 하면? / 커밋·브랜치·역색인을 초기화

기존 이름으로 BRANCH? / Invalid args, 기존 HEAD 보존

INIT 전에 명령 입력? / Run INIT first 안내

대소문자·공백 인자? / 명령 upper, 문자열 shlex로 따옴표 파싱

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 후

BRANCH feature → SWITCH missing → COMMIT " "

확인: 기존 feature가 있으면 Invalid args / Unknown branch: missing / Invalid args

기존 설명 문서에는 BRANCH 덮어쓰기 정책이 남아 있었지만 현재 코드는 중복 이름을 거부합니다. 발표에서는 현재 구현을 기준으로 답합니다. [branch]는 커밋의 생성 당시 메타데이터라 지금 브랜치가 가리키는 곳이라는 의미가 아닙니다. 표준 오류에는 Invalid args, Unknown branch, Unknown commit이 있고, 초기화 전 안내는 별도 확장입니다. 재초기화는 저장 데이터를 없애므로 데모를 다시 시작할 때 사용합니다.

상태를 바꾸는 정책과 출력용 정보를 구분해서 답합니다.

평가 질문: 과제 제약을 지켰나요?

사용한 것

dict: 커밋·브랜치·색인
list: 부모·큐·결과
set: 조상 방문 기록
표준 라이브러리만 사용

사용하지 않은 것

sorted(), list.sort()
그래프 전용 라이브러리
네트워크 통신·파일 내용 추적

선택 과제

Diff·Merge 명령 미구현
복수 정렬 성능 비교 미구현
메모리만 유지, 영속성은 선택

실행 확인 · 명령은 한 줄씩 입력

일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k no_standard_sort
```

확인: 표준 정렬 호출·그래프 전용 라이브러리 사용 여부 검사 · 마지막 OK 확인

AST는 파이썬 코드를 문법 구조로 읽은 것입니다. 주석에 sorted라는 단어가 있어도 실제 호출인지 구분할 수 있습니다. 현재 구현의 호출 AST를 검사하고 import 문도 직접 확인했습니다. 테스트 자체의 무작위 정렬 검증도 표준 정렬 결과에 기대지 않고 오름차순, 원소 보존, 안정성을 확인합니다. 기본 자료형은 허용되므로 dict를 직접 만들 필요는 없습니다.

과제의 목표는 커밋 그래프 알고리즘이며, 파일·네트워크 기능은 범위 밖입니다.

평가 질문: 알고리즘을 어떻게 믿을 수 있나요?

대상 / 독립적인 확인 방법

LOG / 모든 부모의 출력 위치 < 자식 위치

PATH / 작은 그래프의 모든 단순 경로와 비교

ANCESTORS / 별도 스택 탐색의 조상 집합과 비교

정렬 / 오름차순·원소 보존·같은 키의 순서·원본 불변

검색 / 반복 단어·연속 구문·저장소 순회 금지

실제 CLI / main.py 실행, 프롬프트·exit/quit 확인

실행 확인 · 명령은 한 줄씩 입력

일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v
```

확인: Ran 15 tests와 OK 확인. 각 테스트 이름으로 검사 대상 확인

정답 검증용 방식은 제품 알고리즘과 다른 것이 좋습니다. PATH 구현은 BFS를 쓰고, 작은 그래프의 검증은 모든 단순 경로를 열거하여 길이와 문자열을 비교합니다. 이 느린 방식은 작은 테스트에만 사용합니다. 무작위 DAG는 고정 seed를 사용해 다시 같은 입력을 만들 수 있습니다. 과제 예시와는 hash 값이 아니라 흐름과 조건을 비교합니다. 15개 통과가 모든 가능한 입력의 무결함을 증명하는 것은 아닙니다.

구현과 다른 방법으로 기대 결과를 만들고, 같은 결과인지 비교했습니다.

이제 내 말로 1분 동안 설명해 보세요

발표 순서

- ① b3-2에서 python3 main.py로 실행했습니다.
- ② 커밋을 분기하고 LOG·PATH·SEARCH를 확인했습니다.
- ③ 자동 테스트 15개가 통과했고 기록은 제출/results에 있습니다.
- ④ LOG는 부모 우선, PATH는 BFS, 조상은 부모 탐색입니다.
- ⑤ 정렬은 안정 병합 정렬, 검색은 역색인을 직접 구현했습니다.

발표 전 체크

실제 hash로 PATH 입력하기
검증 파일 11과 테스트 소스 열어 두기
Merge 명령과 테스트용 병합 그래프 구분하기

상세 설명

연습 답변: 저는 커밋 메타데이터를 관리하는 Mini Git을 만들었습니다. b3-2에서 실행하고 브랜치를 나누어 부모 관계를 확인했습니다. LOG는 부모가 먼저 나오도록 위상 정렬하고, PATH는 연결을 양방향으로 보아 BFS로 최단 거리를 구합니다. 조상은 부모 방향으로 찾고, 정렬은 병합 정렬, 검색은 역색인으로 구현했습니다. Python 3.12.13에서 테스트 15개가 통과했으며 결과와 코드는 제출/results와 tests 폴더에 있습니다. 질문을 받으면 해당 소스 파일과 결과를 함께 보여 주세요.

실행 명령 하나, 검증 근거 하나, 알고리즘 이유 하나를 꼭 말하세요.